MA342 Project 2

Iayan Ansari & Luke Gaston

March 2026

1 Introduction

Time-delayed differential equations are found everywhere, from biological processes all the way to economics. Despite their added complexity, their ubiquity means that, as Yang Kuang puts it, "to ignore [DDEs] is to ignore reality" [2]. A DDE is an ordinary differential equation whose behavior depends on historical information. This allows faster and more efficient computations for otherwise more complicated ordinary differential equations that would require more extensive applications of numerical methods.

Cloud computing requires an array of multiple computing stations, each with different computation loads, speeds, and capacities, often navigated by a central or primary computer handling the distribution of given computation instructions. This model represents an array of computers in a cloud computing model using historic data in a delayed differential equation.

2 Model

Cloud computing and the processing of data are represented by *packets*, the units of data that are moved between systems. In our model, we represent the distribution of computations by a central computer. This central computer has a processing speed λ that directs packets to a series of sub-level computing units, defined in terms of Q, such as Q_1, Q_2, all the way up to Q_nth level of units, where Q_i defines the number of packets being currently computed in each processing unit, as depicted in Figure 1.

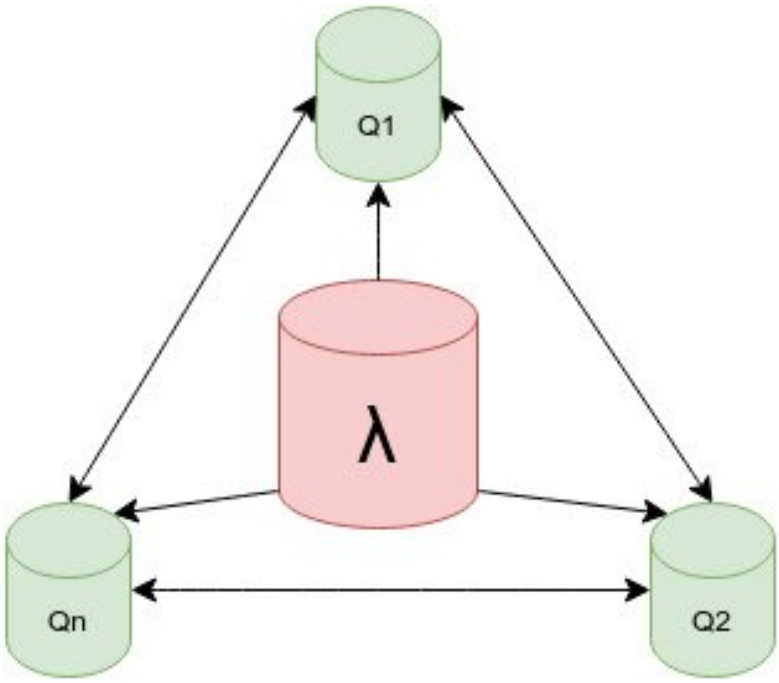


Figure 1: Visual diagram of the cloud computing model. The central server has a packet computing rate, and each "hub" computer has a state of packets computed.

As each processing unit is given packets to compute, the rate at which they are distributed packets is defined by λ, the rate at which each processing unit receives packets from the hub. The efficiency at which these packets are processed is determined by the weight of $1 - f$, which determines the success rate of these packets being properly computed. As the packets are received from the hub to each processing unit, the combined rate at which they are processed is determined by μ. In this model, processing units may send packets in their respective queues to other processing units if under a high load or if other processing units are free of computing and can be utilized. This rate at which packets are transferred between processing units is defined by α, a load-balancing gain that stabalizes the distribution of packets across the network of processing units.

These variables define the rate at which each processing units consumes, transmits, and processes packets. Between the receiving and transfer of packets, time-delays are in effect as transmission is not instantaneous, which is

defined as a time delay of τ. With these terms, our model is defined by the following differential equation.

$$\frac{dQ_i}{dt} = \lambda - \{\mu[1-f]Q_i(t) + \alpha \sum_j [Q_j(t-\tau) - Q_i(t-\tau]\}$$

In this model, Q_i represents the current processing unit and its computation rate being analyzed at a specific time t. The transmission and relation of packets between the surrounding processing units are processed through the summation of Q_j, each external processing unit.

3 Experiments

To investigate the behavior of the cloud computing DDE, a series of controlled experiments was conducted. Each experiment isolated one or two model parameters while holding the others fixed, allowing the effect of each parameter on queue dynamics to be observed independently. The parameters examined were the network delay τ, the load-balancing gain α, the network topology, and the assignment of unique parameter values across individual computers. Additionally, a packet-tracking simulation was developed to qualitatively reproduce the CloudSim experimental results reported by Buyya et al. [1], thereby assessing the validity of the DDE.

The first experiment examined how the lag τ affects the rate at which queue lengths converge to the analytical steady-state queue value. Four delay values were tested — $\tau = 0.5$, 2.0, 5.0, and 10.0 — while the remaining parameters- μ and f- remained constant. Initial queue sizes were set to 20, 5, and 2 to ensure the load-balancing mechanism was active.

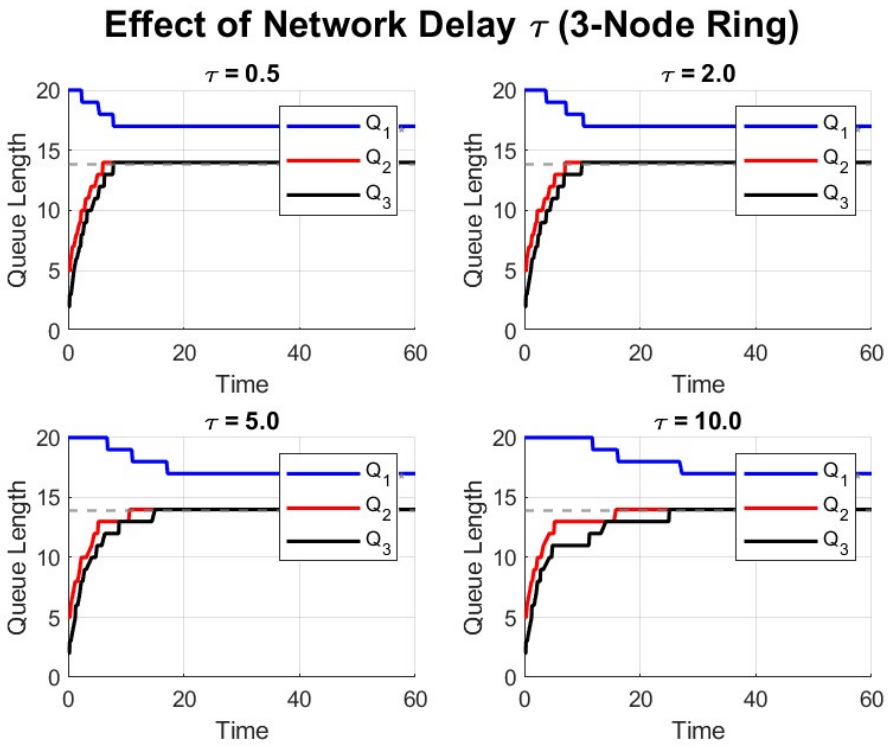


Figure 2: Plots of a 3-Node cloud-computing system with different τ values. Larger τ values correspond to higher settling times.

As seen in Figure 2, larger network delays corresponded to longer convergence times. When the delay is small ($\tau = 0.5$), the computers act on near-current queue information, and all three queues converge rapidly. As τ increases, the load-balancing decisions are based on increasingly outdated queue values, meaning tasks are moved based on queue imbalances that may no longer exist by the time the routed tasks arrive, thus causing the system to stall before eventually settling. At $\tau = 10.0$, the hesitant behavior is most pronounced, and the time required for all queues to converge to Q* is significantly longer. This confirms the initial expectation that outdated information degrades load-balancing performance.

The second experiment examined the role of the load-balancing gain α, which controls how aggressively tasks are redistributed between neighboring computers in response to a detected queue imbalance. Four values were tested — $\alpha = 0.02$, 0.08, 0.20, and 0.50 — with $\tau = 2.0$ and all other parameters

fixed.

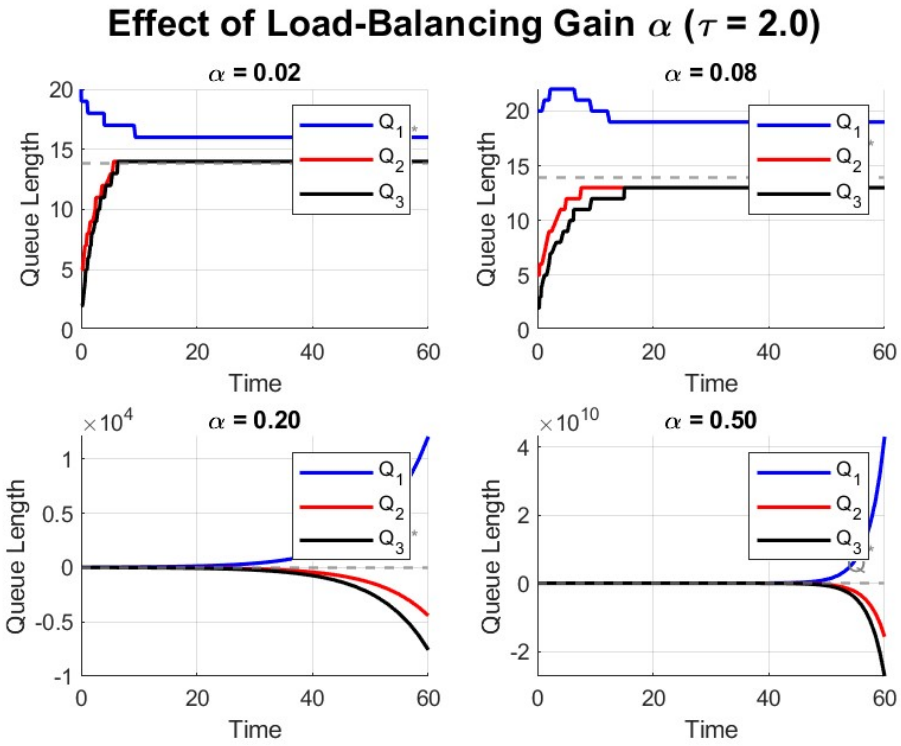


Figure 3: Plots of a 3-Node cloud-computing system with different α values. α values above 0.20 result in divergent, unstable systems.

The key finding is that α is directly tied to queue stability. For small values of α (0.02 and 0.08), all three queues converge smoothly to Q*, with smaller α values converging faster. However, as seen in the bottom subplots of Figure 3, once α exceeded 0.20, the queues diverge; they grow or decay without bound. This instability arises from an aggressive balancing that causes the system to transfer too many packets in response to a perceived imbalance, creating an equal and opposite imbalance on the next cycle. It is concluded that any practical use of this model requires careful tuning of α relative to τ and the network topology, since the stable range of α decreases as the system size and delay increase.

The third experiment sought to extend the delay analysis by assigning a different τ value to each computer rather than using a single system-wide

delay, with four combinations of delays being tested.

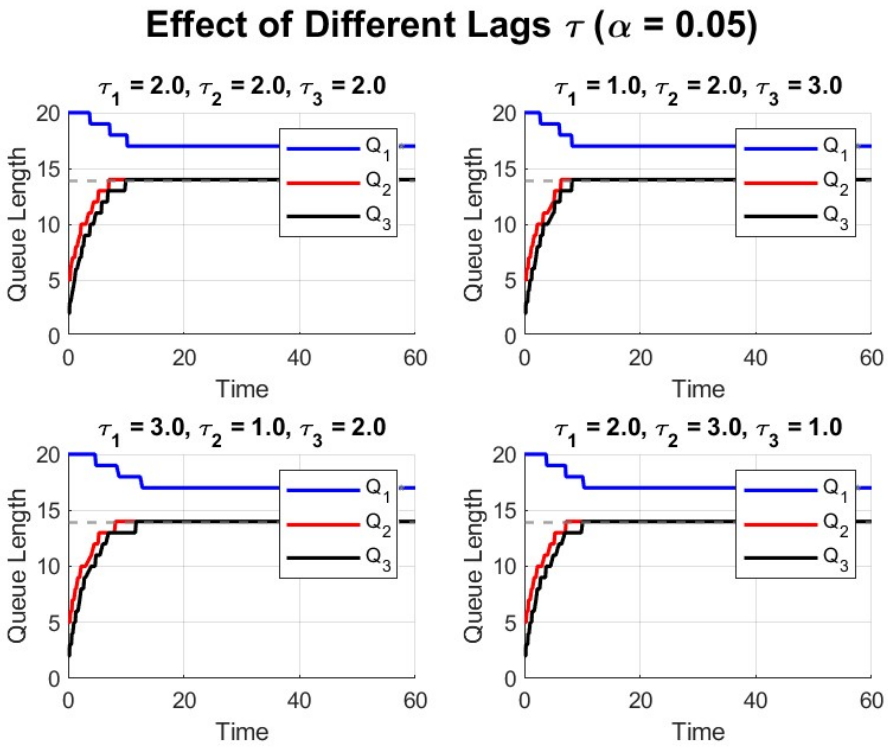


Figure 4: Plots of a 3-Node cloud-computing system with heterogeneous τ values across computers. Average τ values indicate consistent behavior; individual settling times vary across computers and tests.

Figure 4 demonstrates how heterogeneous delays do not produce substantially different behavior compared to a uniform delay of the same magnitude. The queues converge to the steady state value at roughly the same time. The differences between processors — in terms of when they individually reach steady-state — reflect the asymmetry in their respective delays, but the overall settling time of the system remains comparable to the uniform-delay case. This suggests the average network delay is the dominant factor governing convergence and that moderate variations in per-link latency do not fundamentally alter system stability.

The fourth experiment compared four different network topologies: a pentagram (5 nodes), a two-node system, a star (6 nodes with one hub),

and a line (5 nodes). Each topology used a different number of computers and a distinct adjacency structure, allowing the effect of connectivity on load balancing to be assessed.

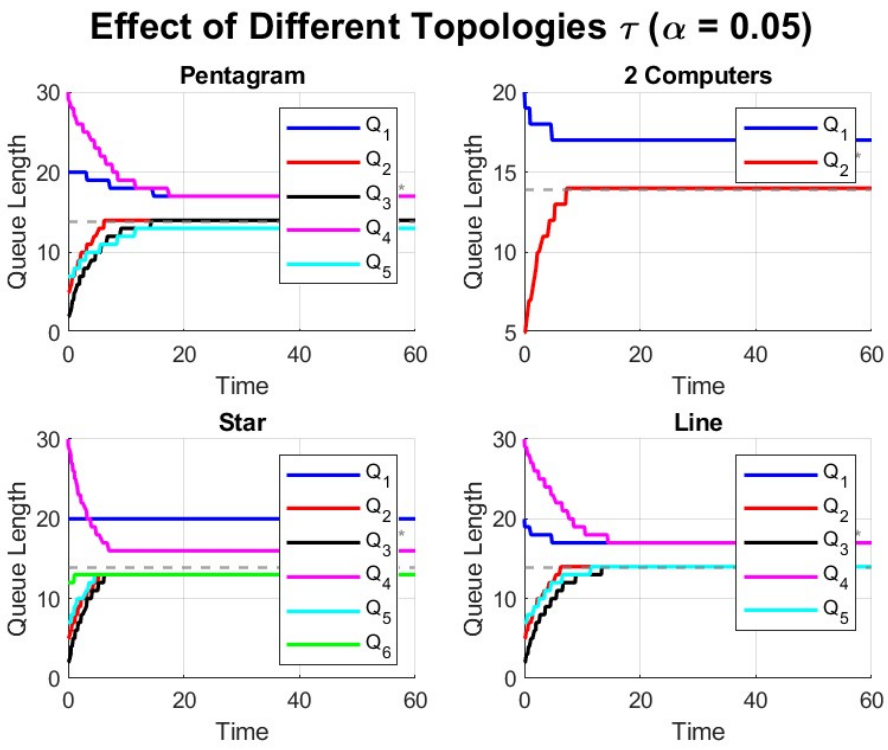


Figure 5: Plots of cloud-computing systems with different topologies. Topologies with fewer intermediaries between high and low-loaded computers have the fastest convergence.

This experiment revealed that topology has a significant effect on both settling time and system capacity. As expected, systems with more computers could handle more total packets. Furthermore, amongst the topologies tested, the star converged the fastest, followed by the two-node system, the line, and finally the pentagram. The star's superior convergence speed is attributable to its hub node. Any imbalance in the network is rapidly communicated to the hub, reducing the number of intermediaries required for packets to reach the least-loaded computer. The line topology, by contrast, forces load-balancing information to propagate sequentially, making it the

slowest to equalize.

The fifth experiment tested whether assigning a different α value to each affects system behavior. Four permutations of per-processor α vectors were applied to a 6-node star topology.

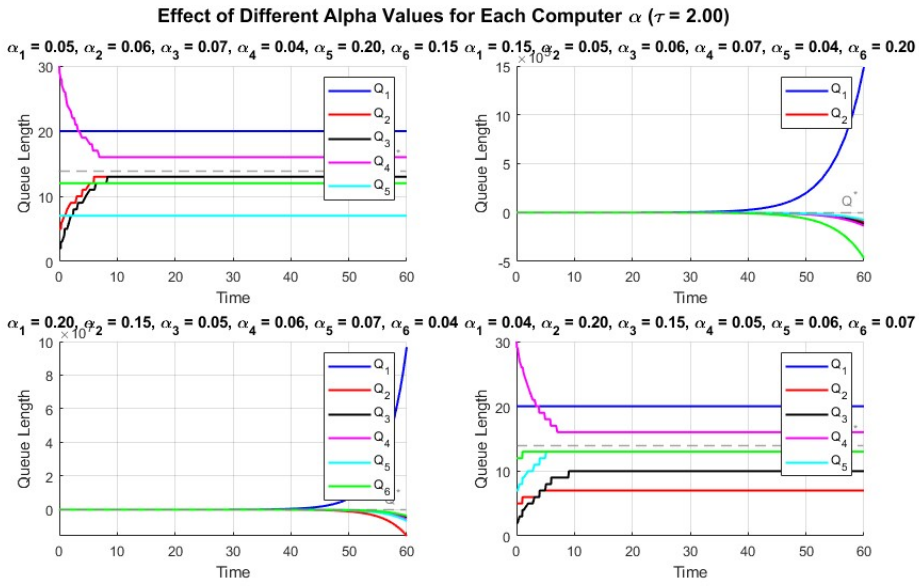


Figure 6: Plots of a star cloud-computing system with heterogeneous α values across each computer. α values have an effect on both overall system settling time and system stability.

The key finding is consistent with the uniform-α experiments in that stability is sensitive to the magnitude of the gain values, and sufficiently large individual α values can destabilize the entire network. The hub node is especially sensitive to α since it participates in load-balancing exchanges with every other node simultaneously. As seen in Figure 6, permutations that assigned the highest α to the hub were the most likely to produce divergent behavior.

The final experiment was to validate the DDE model against the simulation conducted by Buyya et al. [1] in their CloudSim paper. Total tasks, packet arrival rates lambda, and completion rates mu were either taken directly from the paper or calculated based on the info found in the paper. The remaining parameters were approximated. The network topology used was

the fully connected triangle from Figure 11 of the CloudSim paper, which is equivalent to the 3-node ring. The results are shown in Figure 8.

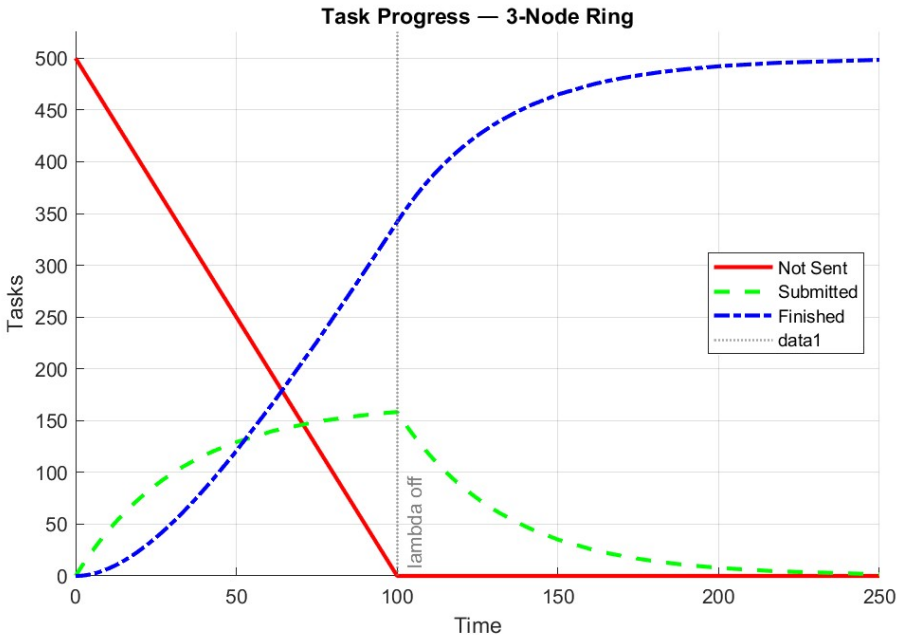


Figure 7: A plot recreation of the cloud-computing system found in the benchmark. The qualitatively similar results shows the formulated DDE can be used to model a more complex discrete-event system.

The qualitative behavior of Figure 7 closely mirrors Figures 9 and 10 in Buyya et al. [1]. The Not Sent curve begins at 500 and decreases linearly to zero at the shutoff time, when all tasks have been injected. The Submitted curve rises from zero as tasks enter the queues, reaches a peak determined by the balance between the arrival rate and the completion rate, and then decays back toward zero as processors drain their queues after injection ceases. The Finished curve is monotonically increasing, beginning near zero and eventually reaching 500 as all tasks are completed. The vertical dotted line marks the moment lambda shuts off, after which the Submitted curve falls and the Finished curve begins to level off. This experiment demonstrates that the DDE framework can reproduce the essential packet life-cycle dynamics captured by a full discrete-event cloud simulator, using only a compact system of delay differential equations.

4 Conclusion

This project developed and analyzed a delay-differential equation to model load balancing in a cloud-computing environment. The model captures the two core physical assumptions: each processor allocates its packets between queue management and task completion, and communication between computers incurs a network delay τ. This DDE formulation allows for the queue sizes to depend on historical queue information rather than on instantaneous values, which more faithfully reflects the information available in a real distribution system.

A series of experiments revealed several key effects each parameter had on the model. First off, network delay is the primary factor governing convergence speed. Larger τ values result in longer settling times because load-balancing decisions are based on increasingly outdated queue data. Secondly, the load-balancing gain α has a stability threshold that depends on both the topology and the lag. Thirdly, network topology significantly affects convergence speed and the stable range of α. Topologies with a dedicated computer for offloading packets, as in the star topology, converge fastest, whereas sparsely connected topologies, such as the line topology, converge more slowly. Fourthly, assigning different τ values to each computer does not fundamentally alter the system's behavior despite introducing asymmetries in individual queue development. Lastly, by recreating the scenario described in Buyya et al. [1], the simulation qualitatively reproduced the task life-cycle curves observed in the CloudSim experimental results, demonstrating that a DDE model can capture the essential dynamics of a large-scale cloud-computing simulation.

In the future, work could be done to study how to model processor failures or heterogeneous completion rates, the effects of incorporating queue-length-dependent lags and exploring optimal control formulations to determine the gain α that minimizes settling time subject to a stability constraint.

References

[1] Rajkumar Buyya, Rajiv Ranjan, and Rodrigo N. Calheiros. Modeling and simulation of scalable cloud computing environments and the cloudsim toolkit: Challenges and opportunities. In *2009 International Conference on High Performance Computing Simulation*, pages 1–11, 2009.

[2] Allen Holder and Joseph Eichholz. *Modeling with Matrices*, page 384. Springer International Publishing, Cham, 2019.